Implementation and measured results

All three milestones implemented and tested on this VM using Mininet, BMv2 simple_switch, P4_16/v1model, and Scapy. These are real data-plane measurements, not simulated estimates.

One trial for each mode, with the same seed-2026 random five tuples and workload: 16 UDP flows, 2,080 packets, and 2,393,600 Ethernet bytes sent per mode.
12 small flows: 20 x 128-byte frames; 2 medium flows: 120 x 512-byte frames; 2 large flows: 800 x 1,400-byte frames. The two large flows dominate the byte total.

Mode	Source bytes	Upper bytes	Lower bytes	Upper %
ecmp	2,393,600	71,680	2,321,920	2.99%
packet	2,393,600	1,261,600	1,132,000	52.71%
flowlet	2,393,600	485,408	1,908,192	20.28%

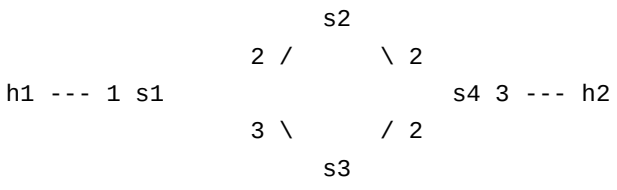
Mode	Received	Inversions	Late packets	Path changes
ecmp	2080	0	0	0
packet	2080	10810	891	0
flowlet	2080	0	0	45

Every mode passed: source bytes = S1 upper + lower bytes; each branch agrees with its intermediate-switch counter; S4 egress agrees with source; all packets arrive exactly once. ECMP has no observed reordering, but is badly imbalanced for this workload. Per-packet random choice improves byte balance and causes reordering. Flowlets avoid observed reordering while permitting path changes; their byte balance still depends on burst sizes and random choices.

Results and raw evidence: results/<mode>/summary.json, sent.json, received.json, switch command logs, and flowlet-register.log.

Milestone 1: topology, ECMP, monitoring

Task 1: topology.json defines two servers h1/h2 and four switches. scripts/run.py builds this topology with explicit port numbers and TCLink delays.



Upper path: S1:2 -> S2:1, S2:2 -> S4:1 (1ms per link). Lower path: S1:3 -> S3:1, S3:2 -> S4:2 (15ms per link). Host links have 0ms configured delay. Nominal forward propagation is 2ms versus 30ms, before processing and queueing.

Task 2: p4/multipath.p4 implements ECMP in balance() at line 45. A CRC32 over source/destination IP, IP protocol, and UDP source/destination ports selects one of ports 2 and 3. The same five tuple always chooses the same port.
first_hop at line 101 is populated only on S1, only for host-facing ingress port 1. The shared program therefore needs no extra first-hop header. S2-S4 use destination LPM forwarding. IPv4 TTL is decremented and the checksum is recomputed. Unknown destinations and unsupported traffic are dropped.

Task 3: port_bytes is a 64-bit register array (line 35). Egress counting at line 126 adds standard_metadata.packet_length to the outgoing port counter. Queries, ARP, and dropped packets are excluded.
answer_query() at line 84 handles EtherType 0x88b5. Request: op=0, port, switch ID, nonce, 64-bit bytes. The addressed switch reads its counter, sets op=1, swaps Ethernet addresses, and sends back toward h1. Queries carry a target ID to avoid ambiguous local replies.
The measurement runner queries every switch/port from h1 only after sender completion and a two-second drain. All counters begin at zero because each experiment starts fresh switches. Command files are generated in commands/; CLI reads only diagnose flowlet path changes, not traffic bytes.

Run: ./run_milestone1.sh. This compiles, creates the topology, configures ECMP, sends the workload, collects in-band counters, validates delivery, and saves measured results.

Milestone 2: byte balance and reordering

Task 1: scripts/workload.py generates seeded random five tuples with different frame sizes and packet counts. scripts/traffic.py sends using Scapy. Each UDP payload begins with HW2!, a 32-bit flow ID, and a 32-bit sequence number starting at zero per flow.
ECMP: 2,393,600 source bytes; 71,680 upper bytes; 2,321,920 lower bytes (2.99% upper). Both large flows hash onto the lower path. Flow counts are therefore a poor proxy for byte balance.
Task 2: mode=1 in balance() draws an independent random bit for each packet, selecting port 2 or 3. This offers each packet a chance to change path; byte equality is not guaranteed for a finite workload.
Per-packet: 2,393,600 source bytes; 1,261,600 upper bytes; 1,132,000 lower bytes (52.71% upper).
Task 3: the receiver records arrival order. scripts/analyze.py counts inversions: pairs i<j with sequence[i]>sequence[j], separately per flow. A merge-based algorithm computes the exact result in O(n log n). Late packets are also reported: arrivals with a sequence below the largest already seen. An inversion is a pair, not a unique packet.
Observed per-packet totals: 10,810 inversions and 891 late packets. ECMP has zero for both. No loss or duplicates confound these results.

Per-packet results for every flow:

Flow	Sent	Recv	Lost	Dup	Inversions	Late packets
0	20	20	0	0	4	4
1	20	20	0	0	6	5
2	20	20	0	0	5	5
3	20	20	0	0	5	5
4	20	20	0	0	6	4
5	20	20	0	0	6	5
6	20	20	0	0	4	3
7	20	20	0	0	3	3
8	20	20	0	0	7	7
9	20	20	0	0	9	8
10	20	20	0	0	3	3
11	20	20	0	0	0	0
12	120	120	0	0	205	46
13	120	120	0	0	197	52
14	800	800	0	0	5264	385
15	800	800	0	0	5086	356

Run: ./run_milestone2.sh. The two modes use the same workload. See results/ecmp/summary.json and results/packet/summary.json for five tuples, all port counters, validation checks, and per-flow counts.

Milestone 3: flowlet switching

Task 1: mode=2 records a valid bit, full CRC32 fingerprint, last ingress timestamp, and selected path in 65,536 register slots. A hash indexes the slot. The first packet chooses randomly; subsequent packets reuse its path until the idle gap exceeds 100,000 microseconds. Then a new random choice is permitted.
BMv2 timestamps are in microseconds. The 100ms threshold exceeds the nominal 28ms path delay difference. The source sends eight bursts per flow and sleeps 250ms between global bursts. Inside a burst, flow packets are interleaved while preserving source order.
Register read/modify/write blocks use @atomic. Fingerprints detect most slot collisions and initialize a new occupant rather than reusing an unrelated path. This finite hash-indexed table is not an exact flow dictionary; collision evictions can disrupt path affinity. Such behavior is a stated limitation, not a general guarantee of ordering.
Task 2: 2,393,600 source bytes; 485,408 upper bytes; 1,908,192 lower bytes (20.28% upper). The diagnostic path-change register recorded 45 changes for existing matching flows.
Flowlet switching produced 0 inversions versus 10,810 for per-packet switching in these trials. Zero observed reordering is consistent with the threshold and pauses chosen here; large queue delays or collisions could invalidate that outcome.

Flowlet results for every flow:

Flow	Sent	Recv	Lost	Dup	Inversions	Late packets
0	20	20	0	0	0	0
1	20	20	0	0	0	0
2	20	20	0	0	0	0
3	20	20	0	0	0	0
4	20	20	0	0	0	0
5	20	20	0	0	0	0
6	20	20	0	0	0	0
7	20	20	0	0	0	0
8	20	20	0	0	0	0
9	20	20	0	0	0	0
10	20	20	0	0	0	0
11	20	20	0	0	0	0
12	120	120	0	0	0	0
13	120	120	0	0	0	0
14	800	800	0	0	0	0
15	800	800	0	0	0	0

Run: ./run_milestone3.sh. All mode configuration is generated into commands/s*-flowlet.txt.
The shared P4 program keeps S2-S4 on destination forwarding.

ECMP per-flow evidence and port counters

ECMP per-flow delivery and reordering:

Flow	Sent	Recv	Lost	Dup	Inversions	Late packets
0	20	20	0	0	0	0
1	20	20	0	0	0	0
2	20	20	0	0	0	0
3	20	20	0	0	0	0
4	20	20	0	0	0	0
5	20	20	0	0	0	0
6	20	20	0	0	0	0
7	20	20	0	0	0	0
8	20	20	0	0	0	0
9	20	20	0	0	0	0
10	20	20	0	0	0	0
11	20	20	0	0	0	0
12	120	120	0	0	0	0
13	120	120	0	0	0	0
14	800	800	0	0	0	0
15	800	800	0	0	0	0

Outgoing data-frame bytes obtained through query packets:

Mode	Switch	Port 1	Port 2	Port 3
ecmp	s1	0	71,680	2,321,920
ecmp	s2	0	71,680	0
ecmp	s3	0	2,321,920	0
ecmp	s4	0	0	2,393,600
packet	s1	0	1,261,600	1,132,000
packet	s2	0	1,261,600	0
packet	s3	0	1,132,000	0
packet	s4	0	0	2,393,600
flowlet	s1	0	485,408	1,908,192
flowlet	s2	0	485,408	0
flowlet	s3	0	1,908,192	0
flowlet	s4	0	0	2,393,600

Byte units include Ethernet/IP/UDP headers and application payload. They exclude Ethernet FCS, preamble, inter-frame gap, and query traffic. The frame is unchanged in length through the fabric, so source and switch totals are directly comparable.

Reproduction, validation, and limitations

Build and unit checks: make all. Requires p4c-bm2-ss plus Python 3; the workload and analysis tests use only the standard library.
Full measurement: sudo -E /path/to/venv/bin/python3 scripts/run.py --mode all. Requires Mininet, Scapy, simple_switch, simple_switch_CLI, tc, ethtool, and privileges to create namespaces and veth links. Use the P4 lab VM with its Python environment activated.
Generate updated report and ZIP: make report; make package. report.py reads saved summaries and refuses to describe experiments with failed checks. package.py includes source, build JSON, topology, commands, raw results, report PDF, and documentation; excludes hidden credentials and caches.
Measurements start fresh switches per mode and explicitly wait for receiver readiness. After sending, a two-second queue drain precedes queries. The runner fails if source/path bytes disagree, intermediate-switch counters disagree, delivery has missing packets, or duplicates appear. Only its own switch and capture processes are stopped in cleanup.
Unit validation checks the inversion algorithm against an independent quadratic oracle on random sequences and checks workload uniqueness, counts, mixed sizes, and repeatability. p4c compiles the program without warnings. End-to-end integration uses the actual P4 JSON on four BMv2 switches.
Limitations: one measured trial per mode; no statistical confidence claim. The source seed reproduces tuples and sizes, but BMv2 random choices and host scheduling can vary. Software-switch queueing and CPU contention can alter balance, delay, and reordering. Flowlet ordering depends on the actual worst-case delay difference, not just nominal link delay.
The educational pipeline supports Ethernet, fixed-header IPv4, and unfragmented UDP. It does not implement ARP, IPv4 options, fragmentation, TCP, arbitrary host applications, or a production forwarding/control plane. Workload source IPs are spoofed to create random five tuples; the receiver records raw packets with Scapy.
Query packets are intended for this isolated lab and have no authentication. The counters are 64-bit and do not saturate. Queries are collected after traffic drains rather than attempting an atomic multi-switch snapshot.

Primary documentation used to verify target behavior:
https://github.com/p4lang/behavioral-model/blob/main/docs/simple_switch.md
https://github.com/p4lang/p4c/blob/main/p4include/v1model.p4

Source files were authored for this assignment; tutorial implementations were not copied. No externally hosted service is needed to run or package this work.